

AFORE: Attention–FFN Disaggregation with Overlapped Reconfiguration of Experts

Wenshuang Li¹, Youhe Jiang², You Peng¹
Jiawei Jiang³, Binhang Yuan¹

¹The Hong Kong University of Science and Technology

²University of Cambridge

³Wuhan University

Abstract

Mixture-of-Experts (MoE) models are increasingly leveraged in deployed large-scale LLM services, but efficient MoE serving remains challenging due to large expert parameters, input-dependent expert activation, and dynamic workloads. Expert parallelism is commonly used to distribute expert computation across GPUs, while attention-FFN disaggregation (AFD) further improves serving efficiency by separating attention and feed-forward computation into independent worker pools. However, we observe that a naive AFD implementation could make expert load imbalance more harmful: once FFN computation becomes an independent pipeline stage, overloaded experts directly slow the FFN stage and degrade end-to-end serving performance.

To solve this problem, we present AFORE, a timely expert reconfiguration system for AFD-based MoE serving. AFORE exploits two architectural properties of AFD. First, AFD exposes the expert-token distribution of upcoming microbatches before they reach FFN execution, enabling placement decisions based on near-future demand instead of stale historical profiles. Second, AFD creates a pipeline window in which expert migration for a target microbatch can be overlapped with the computation of preceding in-flight microbatches. Based on these insights, AFORE formulates expert reconfiguration as a microbatch-aware scheduling problem and uses a migration-aware scheduler to decide when and which experts to migrate. AFORE further implements lightweight demand prefetching and NVLink-based GPU-GPU expert migration to reduce reconfiguration overhead. Evaluation on a 110B-parameter MoE model across four dynamic workloads shows that AFORE improves output throughput by **10.1–17.6%** and reduces P95 inter-token latency by **7.1–9.5%** compared with the strongest competing baseline. Compared with static placement, AFORE improves throughput by **29.8%** on average and reduces P95 inter-token latency by **18.2%** on average. Migration profiling further shows that AFD pipeline overlap can fully hide expert-migration latency.

1 Introduction

Mixture-of-Experts (MoE) models are increasingly used by deployed large-scale large language model (LLM) services, including interactive assistants, coding copilots, search, data analysis, enterprise automation, and agentic applications [5, 8, 9, 14, 17, 29]. By activating only a small subset of experts for each token, MoE models increase model capacity while keeping per-token computation manageable [9, 15]. However, serving MoE inference at scale remains costly: expert parameters are large, expert activation is input-dependent, and workloads change over time [10, 12, 21], where the existing solutions are not sufficient. For example, expert parallelism (EP) distributes experts across GPUs to scale expert execution [21], and attention-FFN disaggregation (AFD) [32] further improves resource utilization by separating attention and feed-forward

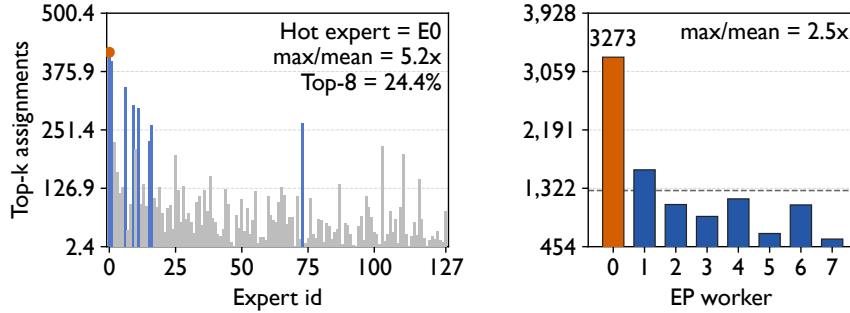


Figure 1 **Left:** Expert routing load in a representative AFD window. With top- $k = 8$, a small set of hot experts receives a disproportionate fraction of routed tokens. **Right:** The resulting load across eight EP workers under static expert placement, where the most loaded worker reaches $2.5\times$ the average load.

computation into distinct worker pools. Yet in AFD, skewed expert routing directly bottlenecks the independent FFN stage, creates pipeline backpressure, and degrades end-to-end serving performance. In this paper, we explore *how to reduce dynamic expert-load imbalance in AFD-based MoE serving without incurring reconfiguration overhead that erodes the resulting performance gains*.

Addressing this problem is essential because the efficiency of MoE serving increasingly determines the cost and responsiveness of production LLM services [11–13, 21]. As modern MoE models grow and workloads become more heterogeneous, small inefficiencies in expert execution can translate into substantial GPU underutilization, reduced output throughput, and higher tail latency [4, 11, 13, 14, 17, 29]. AFD makes this issue especially consequential: once FFN computation is isolated as a separate pipeline stage, imbalance among expert workers can throttle the entire serving pipeline rather than remaining a localized execution inefficiency [2, 23, 25, 32]. Addressing FFN-side expert imbalance is therefore essential for realizing the scalability benefits of AFD while preserving the latency guarantees required by interactive LLM applications [2, 11, 13, 25, 32].

The key system obstacle is that the existing techniques for scaling MoE serving improve capacity and resource utilization but *do not remove the dynamic imbalance that arises from input-dependent expert routing*. First, EP partitions expert parameters and computation across GPUs, enabling large expert layers to exceed the memory and compute capacity of a single device [11–13, 21]; however, such an approach also makes expert placement and resource allocation central to performance, because GPUs hosting popular experts become stragglers while others remain underutilized. This imbalance is difficult to avoid statically: different requests, layers, and generation phases can activate substantially different experts, producing the skewed expert-load distributions shown in Figure 1 and across-layer imbalance shown in the left panel of Figure 2.

Second, AFD provides a complementary architecture by separating attention and FFN computation into independently provisioned worker pools [2, 23, 25, 32], but this separation makes the FFN/expert stage a distinct pipeline bottleneck rather than merely one component of monolithic execution. To examine the performance impact of routing skew, we compare synthetic uniform and skewed routing under monolithic attention-FFN execution and AFD. As shown in the right panel of Figure 2, skewed routing reduces normalized throughput by 11.9% under monolithic execution and by 20.1% under AFD. Thus, existing scalability mechanisms address model capacity and coarse resource provisioning, but they are insufficient for handling rapidly changing expert-load skew: an overloaded FFN worker in AFD slows the independent FFN stage, creates pipeline bubbles, backpressures the attention stage, and degrades end-to-end serving performance.

A natural approach to mitigating expert load imbalance is by reconfiguration, which adjusts expert placement according to runtime load [11, 22, 27]. Reconfiguration can improve expert utilization and reduce stragglers, but existing reconfiguration mechanisms face two fundamental challenges:

- **C1: Dynamic and unpredictable demand.** Expert demand changes over time because token routing depends on the input workload, model layer, sequence position, and request mix [4, 13]. Static placement, historical statistics, or coarse-grained profiling can capture only average or past expert popularity; they cannot reliably describe the expert-token distribution of upcoming execution [11, 18]. As a result,

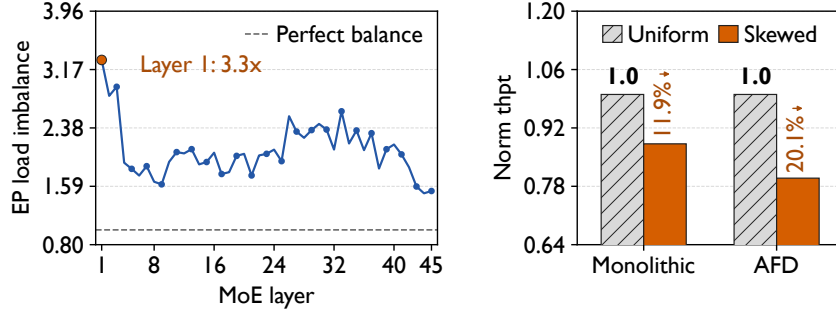


Figure 2 **Left:** EP load imbalance across MoE layers on ShareGPT, with the most imbalanced layer reaching $3.3\times$ imbalance of the average worker load. **Right:** Normalized throughput under synthetic uniform and skewed routing. Skewed routing reduces throughput by 11.9% under monolithic attention-FFN execution and by 20.1% under AFD.

reconfiguration decisions may react too late or optimize for stale load patterns.

- **C2: Non-negligible system overhead.** Expert reconfiguration introduces runtime overhead from placement scheduling, expert-weight migration, routing-metadata updates, and worker coordination [22, 27]. If these operations are exposed on the critical path, the cost of reconfiguration can offset its load-balancing benefit and even degrade overall performance [1, 24].

Our key observation is that AFD, while making expert imbalance more harmful, also provides two architectural opportunities for addressing these challenges. *In particular, AFD both amplifies expert imbalance and exposes the temporal structure needed to fix it.*

- **O1: Microbatch-level demand is visible.** AFD processes requests by a disaggregated microbatch pipeline, where a target microbatch is handled by the attention side before reaching the FFN/expert side [32]. This pipeline structure exposes the expert-token distribution of the target microbatch before its FFN execution. Therefore, reconfiguration decisions can use near-future demand already available in the pipeline, enabling timely expert placement adaptation under dynamic workloads (addressing C1).
- **O2: reconfiguration overhead can be overlapped.** AFD creates a pipeline window between observing the target microbatch’s expert demand and executing that microbatch on the FFN/expert side [23, 32]. During this window, preceding in-flight microbatches continue executing on FFN workers. This pipeline structure allows expert migration for the target microbatch to overlap with preceding microbatch computation, reducing the migration overhead exposed on the critical path (addressing C2).

Based on these insights, we present AFORE to accelerate MoE inference serving by attention-FFN disaggregation with overlapped reconfiguration of experts. AFORE adapts expert placement over time using microbatch-level expert-demand information and overlaps expert migration with the computation of preceding in-flight microbatches. At its core, AFORE treats expert reconfiguration as a microbatch-aware scheduling problem: for each target microbatch, the system estimates the FFN-stage imbalance under the current placement, evaluates candidate expert-placement changes, and triggers migration only when the expected reduction in FFN bottlenecks outweighs the migration overhead exposed on the critical path. This design improves FFN-stage balance and end-to-end serving efficiency under dynamic workloads. Our contributions are summarized as follows:

Contribution 1: We formulate temporal expert reconfiguration in AFD-based MoE serving as an online expert-placement scheduling problem. The problem jointly determines when to adjust expert placement and which experts to migrate for upcoming microbatches, while accounting for both near-future expert demand and exposed migration overhead. To solve this problem efficiently, we design a migration-aware scheduling algorithm that uses prefetched expert-token distributions to estimate FFN-stage imbalance, evaluates the benefit of candidate placement changes, and performs reconfiguration only when it reduces the critical-path cost of FFN execution.

Contribution 2: We design and implement AFORE with runtime support for microbatch-level demand prefetching and efficient expert migration. AFORE introduces a lightweight prefetching interface that collects the expert-token distribution of target microbatches before they reach FFN execution, enabling placement decisions based on near-future demand. AFORE overlaps expert migration with the computation of preceding in-flight microbatches and applies placement updates at microbatch boundaries to avoid disrupting ongoing execution. To reduce migration time, AFORE uses NVLink-based GPU-GPU expert transfer for fast expert reconfiguration.

Contribution 3: We evaluate AFORE on a 110B-parameter MoE model (i.e., GLM-4.5-Air) under four dynamic serving workloads and compare AFORE against static placement, statistics-based reconfiguration, reactive load-balancing, and prediction-based reconfiguration baselines. Experimental results show that AFORE improves output throughput by **10.1–17.6%** and reduces P95 inter-token latency by **7.1–9.5%** compared with the strongest competing baseline. Component-wise ablation shows that AFORE improves throughput by **29.8%** on average and reduces P95 inter-token latency by **18.2%** on average over static placement. Migration profiling further shows that AFD pipeline overlap can fully hide expert-migration latency.

The rest of this paper is organized as follows. §2 provides background and related work. §3 presents the motivation for temporal expert reconfiguration in AFD-based MoE serving. §4 introduces the overall architecture of AFORE. §5 presents the scheduling framework for expert reconfiguration. §6 describes the system design and runtime implementation. §7 presents the system evaluation. §8 concludes the paper.

2 Background and Related Work

Expert parallelism for MoE deployment. MoE models improve the parameter efficiency of LLMs by activating only a small subset of experts for each token [5, 8, 9, 14, 15, 17, 29]. This sparse activation enables larger parameter space without a proportional increase in per-token computation, which also makes serving more complex because expert parameters and expert computation must be distributed across devices. Recent MoE serving systems commonly rely on expert parallelism to partition experts across GPUs and dispatch tokens to the corresponding expert workers [12, 13, 21]. For instance, FastMoE [12] and Tutel [13] optimize distributed expert execution and token dispatch; DeepSpeed-MoE [21] provides system support for large-scale MoE training and inference; and recent large MoE models further increase the need for efficient expert-parallel serving [14, 17, 29]. These systems establish expert parallelism as a standard mechanism for scaling MoE deployment. However, expert-parallel performance depends heavily on expert placement and resource allocation, since uneven expert activation could lead to serious stragglers. Our work builds on expert-parallel MoE serving and studies how expert placement should be adapted when the FFN/expert stage is deployed under AFD.

Attention-FFN disaggregation. A separate thread of effort improves LLM serving by disaggregating computation components with different resource requirements [2, 23, 25, 32]. In AFD, attention and feed-forward computation are placed on separate worker pools so that each side can be provisioned and optimized independently. This design is motivated by the different bottlenecks of the two stages: attention execution is bounded by KV-cache and other relevant memory access, while FFN execution is dominated by dense or expert MLP computation. Recent AFD systems study how to organize the disaggregated pipeline, transfer intermediate activations, and allocate resources between attention and FFN workers [2, 25, 32]. These works show that AFD is a promising architecture for scalable LLM serving. Our work focuses on maintaining efficient MoE serving under the AFD architecture, where the FFN side may contain expert-parallel computation and its load balance directly affects the service rate of the disaggregated pipeline.

MoE workload dynamics and expert-load imbalance. Expert-load imbalance is an essential challenge in MoE serving systems because token routing is input-dependent [4, 9, 11, 13, 15]. Different requests, layers, and generation phases can activate different subsets of experts, causing some experts to receive far more tokens than others. Prior work addresses this issue from several angles [26, 28]. MoE model designs use auxiliary load-balancing losses or routing constraints to encourage more uniform expert usage [9, 15]; systems such as Tutel optimize token dispatch and communication under skewed activation [13]; and expert-placement or prediction methods exploit routing statistics to reduce load skew and communication overhead [4, 11, 18].

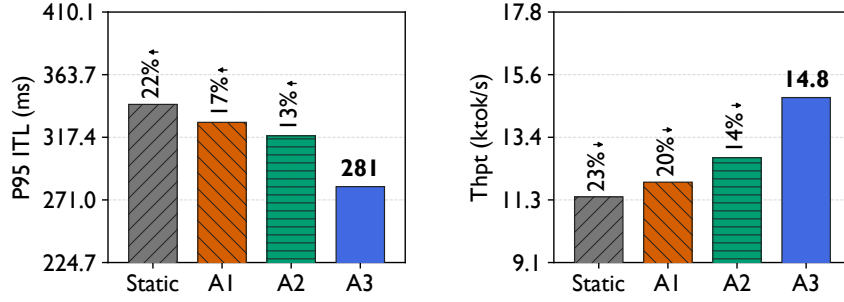


Figure 3 Left: P95 inter-token latency under static baseline placement and the three progressively enabled reconfiguration approaches. Right: Corresponding output throughput under the same configurations.

These techniques reduce imbalance, but they generally rely on training-time regularization, static placement, or historical runtime feedback. In contrast, AFORE targets serving-time expert-load dynamics under AFD by leveraging the disaggregated microbatch pipeline to make expert-placement decisions with timely workload information.

MoE expert reconfiguration. Expert reconfiguration dynamically changes expert placement, replication, or resource allocation to adapt to runtime load [1, 11, 18, 22, 24, 27]. For example, placement-oriented systems optimize the mapping between experts and GPUs to reduce load imbalance and communication cost [11]; elastic MoE systems adjust expert resources under changing workloads [22, 27]; and online balancing mechanisms migrate or redistribute experts to avoid persistent hot spots [18, 24]. These approaches demonstrate the value of adapting expert resources at runtime. However, reconfiguration also introduces overhead that can offset the benefit of load balancing and even degrade end-to-end performance. AFORE therefore treats reconfiguration not only as a placement problem, but also as a scheduling problem: it overlaps expert migration with the computation of preceding in-flight microbatches in the AFD pipeline, reducing the reconfiguration overhead exposed to serving.

3 Case Study and Motivation

We first present a controlled case study to understand how expert reconfiguration affects AFD-based MoE serving, where we focus on three key questions:

- **Question 1:** *whether static expert placement is sufficient under dynamic expert demand.*
- **Question 2:** *whether microbatch-level demand prefetching can improve reconfiguration decisions.*
- **Question 3:** *whether the pipeline execution of AFD can hide expert migration overhead.*

Experiment setup. We serve a 110B-parameter MoE model (GLM-4.5-Air) on a cluster of 16 NVIDIA A100 GPUs under a PD+AFD deployment. The cluster is partitioned into 4 prefill workers, 4 attention/routing workers, and 8 FFN/expert workers; the FFN side uses expert parallelism degree 8. We replay a dynamic serving workload in which the expert-token distribution varies across microbatches. All configurations use the same model, workload, GPU allocation, batching policy, and FFN expert-parallel layout. We compare four configurations: (i) static expert placement, (ii) expert reconfiguration without prefetch or overlap, (iii) expert reconfiguration with prefetch but without overlap, and (iv) expert reconfiguration with prefetch and overlap. We report end-to-end throughput and inter-token latency in Figure 3.

Baseline approach: static expert placement. We first evaluate static expert placement, where experts are evenly partitioned across the 8 FFN workers at initialization, and the placement remains fixed throughout serving. This configuration introduces no migration overhead, but it cannot adapt to changes in expert demand dynamics. As shown in Figure 3, the baseline reaches 11,357.31 tokens/s throughput and 341.713 ms P95

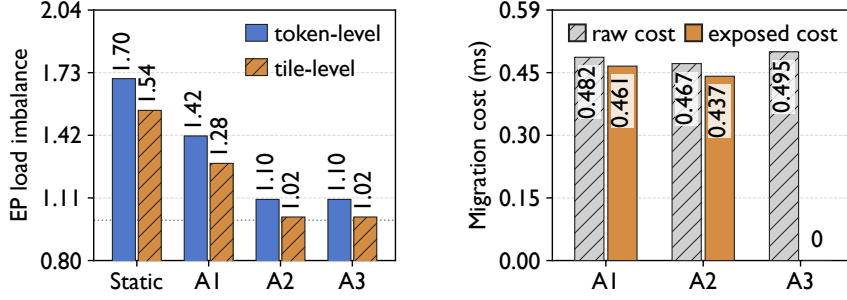


Figure 4 Left: EP load imbalance ratio under Static and Approaches 1–3 (mean values). Right: Copy-readiness latency and exposed wait under Approaches 1–3 (mean values).

inter-token latency. Its mean token-assignment EP load imbalance ratio is $1.70\times$, causing the FFN stage to wait for overloaded workers.

Approach 1: expert reconfiguration without prefetch or overlap. We next evaluate dynamic expert reconfiguration without using demand prefetching or migration overlap. For a target microbatch m_t in FFN execution order, let d_t denote its per-expert demand and $d_{h(t)}$ the demand observed at the same layer and microbatch position in the preceding serving step. This approach uses $d_{h(t)}$ rather than d_t to identify historical hot experts and adjust their placement. When a reconfiguration decision is made, the system migrates the selected experts and applies the new placement before executing m_t . Compared with Static, this approach lowers the mean token-assignment EP load imbalance ratio from $1.70\times$ to $1.42\times$ (a 16.7% reduction), improves throughput by 4.5%, and reduces P95 inter-token latency by 3.9%.

Insight 1: dynamic reconfiguration is necessary. The improvement over Static shows that expert placement should not remain fixed under dynamic MoE workloads. Even a simple history-based policy can reduce FFN-stage stragglers by moving hot experts away from overloaded workers. This confirms that expert reconfiguration is necessary for maintaining FFN-stage balance in AFD-based MoE serving.

Approach 2: expert reconfiguration with prefetch but without overlap. We then isolate the benefit of microbatch-level demand prefetching. Instead of relying on historical demand $d_{h(t)}$, this approach uses the target microbatch’s actual demand d_t to guide expert placement. Compared with Approach 1, it lowers the mean token-assignment EP load imbalance ratio from $1.42\times$ to $1.10\times$ (a 22.2% reduction), as shown in the left panel of Figure 4. It also improves throughput by 7.1% and reduces P95 inter-token latency by 3.0%.

Insight 2: prefetching improves reconfiguration quality. History-based reconfiguration can reduce persistent imbalance, but it may still adapt to stale load patterns when expert demand changes quickly. Microbatch-level prefetching exposes the actual expert demand of upcoming execution, allowing the approach to choose placements that better match the workload about to enter the FFN stage. This validates O1 in §1.

Approach 3: expert reconfiguration with prefetch and overlap. Finally, we evaluate expert reconfiguration with both demand prefetching and migration overlap. This approach uses the same prefetched expert-token distribution as Approach 2, but starts expert migration as soon as the target microbatch’s demand becomes available. The migration overlaps with the FFN-stage computation of preceding in-flight microbatches; the target uses the new placement after the required copies complete. Compared with Approach 2, this approach reduces mean exposed migration wait by 100%, improves throughput by 16.4%, and reduces P95 inter-token latency by 11.8%. Overall, the complete design improves throughput by 30.3% and reduces P95 inter-token latency by 17.8% over Static. The right panel of Figure 4 compares mean copy-readiness latency with mean exposed wait. The mean exposed waits for Approaches 1–3 are 0.461, 0.437, and 0.000 ms, respectively.

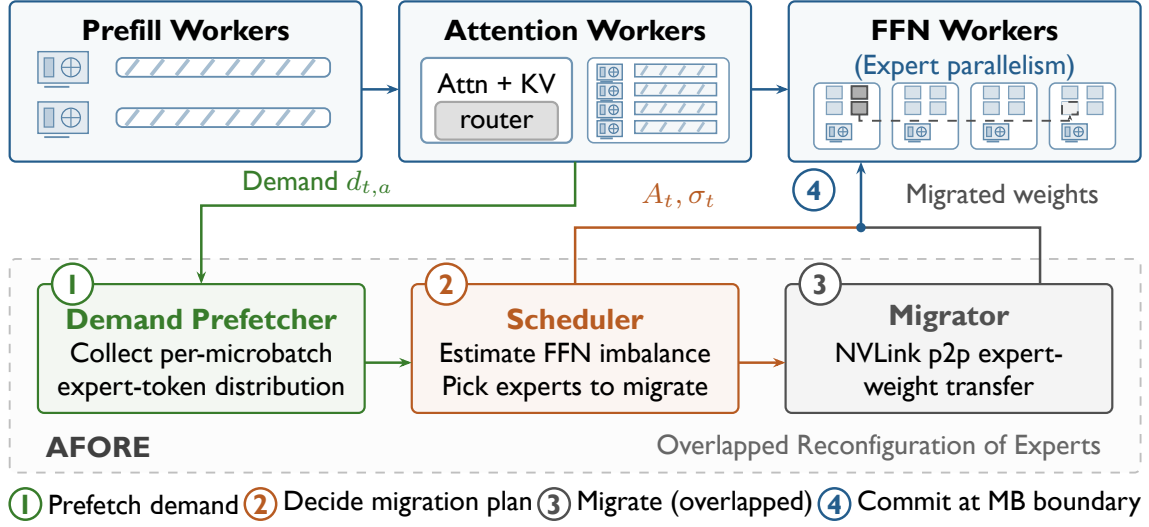


Figure 5 Overall architecture of AFORD. AFORD adds a layer for overlapped reconfiguration of experts between attention and FFN workers to prefetch upcoming expert demand, schedule migration-aware placement updates, overlap expert migration with in-flight FFN execution, and commit new placements at microbatch boundaries.

Insight 3: pipeline overlap reduces reconfiguration cost. Prefetching improves the quality of placement decisions, but migration overhead can compromise the benefit of reconfiguration if it is fully exposed on the critical path. AFD provides a pipeline window between observing a target microbatch’s expert demand and executing that microbatch on the FFN side. By overlapping expert migration with the computation of preceding in-flight microbatches, the system can hide some migration overhead and improve overall utilization. This validates [O2](#) in §1.

Implications for system design. The case study leads to three design implications. First, dynamic expert reconfiguration is necessary because static expert placement cannot maintain FFN-stage balance under changing expert demand. Second, AFD’s microbatch-level demand prefetching benefits reconfiguration by exposing the exact near-future expert demand before FFN execution. Third, AFD’s microbatch pipeline enables expert migration to overlap with preceding microbatch computation, further improving system utilization by reducing exposed migration overhead. We design AFORD based on these implications: AFORD formulates expert placement as a microbatch-aware scheduling problem, uses demand prefetching to guide reconfiguration, and schedules fast expert migration within the AFD pipeline window.

4 AFORD Overview

This section presents the overall architecture of AFORD. As shown in [Figure 5](#), AFORD is built on top of an AFD-based MoE serving runtime. The system contains three major components: the *demand prefetcher*, the *placement scheduler*, and the *placement migrator*. Together, these components allow AFORD to observe upcoming expert demand, determine whether expert placement should be adjusted, and perform expert migration with minimal impact on the AFD pipeline.

Overall routine. AFORD serves MoE inference through separate prefill, attention, and FFN workers. We introduce a layer for overlapped reconfiguration of experts between the attention side and the FFN side, which continuously monitors upcoming expert demand and adapts expert placement before target microbatches start FFN execution. The runtime workflow of AFORD proceeds as follows. First, when the attention workers process a target microbatch, the demand prefetcher extracts its expert-token distribution before the microbatch reaches the FFN side. Second, the placement scheduler uses this prefetched information, together with the current expert placement and FFN worker status, to estimate whether the target microbatch will create FFN-stage

Table 1 Notation in the scheduling problem formulation.

Symbol	Meaning
$\mathcal{W}, N$	set of FFN expert workers, and their count
$\mathcal{E}, E$	set of MoE experts, and their count
w, e	a worker index/ an expert index
m_t	target microbatch at scheduling epoch t
d_t^e	token-expert assignments of m_t routed to e
$S_t = \sum_e d_t^e$	total token-expert assignments in m_t
$P = (x_{e,w})$	placement; $x_{e,w}=1$ iff e has a replica on w
$\mathcal{R}_e(P), r_e(P)$	replica set and replication degree of e
K_w	expert-slot (memory) budget of worker w
$A_t = (a_t^{e,w})$	integer assignment counts for m_t across workers
$\phi_{e,w}$	fraction of e 's tokens routed to its replica on w
$L_w(A_t)$	token load of worker w
M_{tile}	grouped-GEMM M-dimension tile size
$T_w(A_t)$	grouped-GEMM tile load of worker w

imbalance. If reconfiguration is beneficial, the scheduler selects the experts to migrate and determines the target placement. Third, the placement migrator executes the selected migration while preceding in-flight microbatches continue computing on FFN workers. Finally, once migration completes, AFORE commits the new placement at a microbatch boundary, so the target or subsequent microbatches can use the updated expert placement without disrupting in-flight execution.

5 Scheduling Framework

In this section, we present the scheduling framework of AFORE. We first formulate the temporal expert reconfiguration problem under AFD-based MoE serving, and then introduce the migration-aware scheduling algorithm that determines when and how to adjust expert placement. Table 1 summarizes the notation used in this section.

5.1 Problem Formulation

We formulate AFORE scheduling as an online, microbatch-driven MoE expert placement problem for an AFD-based MoE serving system. On the FFN side, a set of expert workers execute in expert parallelism, with each worker hosting a subset of the MoE experts. AFORE proceeds in scheduling epochs. At each epoch, the scheduling algorithm first observes the expert-token demand of an upcoming target microbatch, and then decides whether to retain the current placement or install a new one by migrating expert replicas across workers. The objective is to improve end-to-end AFD performance by relieving the FFN-side bottleneck, while accounting for the overhead that expert migration introduces.

Modeling system execution and expert placement. The FFN side comprises N expert workers $\mathcal{W} = \{1, \dots, N\}$ that jointly host E MoE experts $\mathcal{E} = \{1, \dots, E\}$. Requests are processed as a stream of microbatches $\dots, m_{t-1}, m_t, m_{t+1}, \dots$ in execution order, and a scheduling epoch is associated with each target microbatch m_t . After the attention/routing side finishes m_t , the prefetcher (§4) exposes its expert-token demand $d_t = (d_t^1, \dots, d_t^E)$, where $d_t^e \in \mathbb{Z}_{\geq 0}$ is the number of tokens routed to expert e . An expert placement is a binary assignment $P = (x_{e,w})$, where $x_{e,w} = 1$ iff a replica of expert e resides on worker w ; we write $\mathcal{R}_e(P) = \{w : x_{e,w} = 1\}$ for the replica set of e and $r_e(P) = |\mathcal{R}_e(P)|$ for its replication degree. A high-load expert may be placed on multiple workers ($r_e > 1$) so that its tokens are served in parallel, whereas a cold expert needs only a single replica. A placement is feasible iff every expert has at least one replica and no

worker exceeds its device-memory budget of K_w expert slots:

$$r_e(P) \geq 1 \quad \forall e \in \mathcal{E}, \quad \sum_{e \in \mathcal{E}} x_{e,w} \leq K_w \quad \forall w \in \mathcal{W}. \quad (1)$$

FFN load and scheduling objective. Let $A_t = (a_t^{e,w})$ denote the work assignment, where $a_t^{e,w} \in \mathbb{Z}_{\geq 0}$ counts the token–expert assignments executed by worker w for expert e . Each expert’s demand is distributed among its available replicas:

$$\sum_{w \in \mathcal{W}} a_t^{e,w} = d_t^e \quad \forall e \in \mathcal{E}, \quad a_t^{e,w} = 0 \quad \text{if } x_{e,w} = 0. \quad (2)$$

The corresponding routing fraction is $\phi_{e,w} = a_t^{e,w}/d_t^e$ when $d_t^e > 0$, and zero otherwise. The token load of worker w is

$$L_w(A_t) = \sum_{e \in \mathcal{E}} a_t^{e,w}. \quad (3)$$

Token load alone does not fully describe grouped-GEMM work. The kernel processes each expert’s assignments in M-dimension tiles, and assignments of different experts cannot share a tile. We define the tile load of worker w as

$$T_w(A_t) = \sum_{e: a_t^{e,w} > 0} \left\lceil \frac{a_t^{e,w}}{M_{\text{tile}}} \right\rceil, \quad (4)$$

where M_{tile} is the M-dimension tile size used by the grouped-GEMM kernel. Ignoring migration cost, the scheduling objective is to minimize the maximum worker tile load:

$$\min_{P, A_t} \max_{w \in \mathcal{W}} T_w(A_t), \quad (5)$$

subject to the placement and assignment constraints in Equation 1 and Equation 2. In practice, changing the placement is not free, so AFORE must weigh this benefit against the cost of migration.

Migration cost. Switching from the current placement P_{t-1} to a new placement P_t requires transferring the weights of every replica that P_t adds (removing a replica requires no weight transfer):

$$\Delta(P_{t-1} \rightarrow P_t) = \{(e, w) : x_{e,w}^t = 1 \wedge x_{e,w}^{t-1} = 0\}. \quad (6)$$

Transferring these weights incurs a migration cost $M(\Delta)$ (its concrete realization is in §6). Because AFD exposes the target microbatch before it reaches the FFN side, this migration can be overlapped with the in-flight microbatches still executing on FFN workers; only the part that exceeds the available overlap window W_t is exposed on the critical path:

$$O_t(\Delta) = \max(0, M(\Delta) - W_t). \quad (7)$$

The exposed cost O_t , not the raw migration cost, is what a placement change contributes to the optimization objective.

Migration-aware placement problem. The load-balancing objective (Equation 5) and the migration cost (Equation 7) compete: a new placement can lower the FFN load, but only by paying to migrate experts. AFORE combines them into a single objective. We estimate the FFN execution time of an assignment as $F(A_t) = \beta_{\text{tile}} \max_w T_w(A_t)$, where β_{tile} is the estimated computation time per tile. Let A_t^{base} be the assignment for the current target m_t after rebalancing among the existing replicas in P_{t-1} . The net gain of a candidate placement and assignment (P_t, A_t) is

$$G_t(P_t, A_t) = F(A_t^{\text{base}}) - F(A_t) - O_t(\Delta(P_{t-1} \rightarrow P_t)). \quad (8)$$

Algorithm 1 AFORE Per-Microbatch Scheduling

Input: target microbatch m_t , demand d_t , projected slot table C_t , protected slots U_t , previous assignment A_{t-1} , global copy budget B , compactness threshold $\theta \geq 0$, cost estimates β_{tile} , $M(\cdot)$, W_t

Output: microbatch plan Π_t

```
▷ Stage 1: Assignment initialization
1:  $P \leftarrow \text{fixed primaries} \cup \text{replica bindings recorded in } C_t$ ;  $\Delta_t \leftarrow \emptyset$ ;  $b \leftarrow 0$ 
2:  $(N_t, T) \leftarrow \text{InitializeAssignment}(A_{t-1}, d_t, P; \theta)$ 
▷ Stage 2: Rebalancing within the placement
3: while FindWithinMove( $N_t, P, T; \theta$ ) returns  $(e, h, c, q)$  with  $q > \theta$  do
4:   move  $q$  tiles of  $e$  from  $h$  to  $c$ ; update  $(N_t, T)$ 
5: end while
▷ Stage 3: Rebalancing beyond the placement
6:  $F_0 \leftarrow \beta_{\text{tile}} \max_w T_w$ ;  $g^* \leftarrow 0$ 
7: save  $(P, N_t, C_t, \Delta_t)$  as the selected candidate
8: while  $b < B$  and FindBeyondMove( $N_t, P, T, C_t, U_t; \theta$ ) returns  $(e, h, c, r, q)$  with  $q > \theta$  do
9:   rebind slot  $r$  to  $e$  on  $c$ ; update  $(C_t, P)$ 
10:  add  $(e, c, r)$  to  $\Delta_t$ ;  $b \leftarrow b + 1$ 
11:  move  $q$  tiles of  $e$  from  $h$  to  $c$ ; update  $(N_t, T)$ 
12:   $g \leftarrow F_0 - \beta_{\text{tile}} \max_w T_w - O_t(\Delta_t)$ 
13:  if  $g > g^*$  then
14:    save  $(P, N_t, C_t, \Delta_t)$  as the selected candidate
15:     $g^* \leftarrow g$ 
16:  end if
17: end while
18: restore the selected  $(P, N_t, C_t, \Delta_t)$ 
19: derive  $A_t$ ,  $\phi$ , and  $\sigma_t$  from  $N_t$  and projected slots
20: record projected state; return  $\Pi_t = (P, A_t, \sigma_t, \Delta_t)$ 
```

At each epoch t , AFORE selects a placement and assignment that maximize this net gain:

$$(P_t^*, A_t^*) = \arg \max_{P_t, A_t} G_t(P_t, A_t), \quad (9)$$

subject to Equation 1 and Equation 2. The selected pair is installed only when $G_t(P_t^*, A_t^*) > 0$; otherwise AFORE retains P_{t-1} with assignment A_t^{base} . This criterion captures the central decision of AFORE: reconfigure only when the FFN time saved by rebalancing outweighs the migration cost it exposes.

5.2 Scheduling Algorithm

Searching the full placement space in Equation 9 for every microbatch is impractical. AFORE therefore fixes each expert’s primary placement and uses the remaining capacity on each worker as *redundant slots*. Scheduling determines which replicas occupy these slots and how to distribute work across the resulting placement. Existing replicas can be reused across microbatches, reducing the need for new weight copies.

The scheduler uses a greedy search in three phases. *Assignment initialization* adapts the preceding assignment to the target demand. *Rebalancing within the placement* first redistributes work over existing replicas without adding weight copies. *Rebalancing beyond the placement* then explores new replicas and selects a candidate only when its FFN time savings outweigh the exposed migration cost. This order exploits the current placement before paying to change it. The scheduler returns $\Pi_t = (P, A_t, \sigma_t, \Delta_t)$, where P is the selected placement, A_t gives the assignment counts across workers, $\sigma_t(e, w)$ identifies expert e ’s physical slot on worker w , and Δ_t lists the required new weight copies. Algorithm 1 summarizes the procedure.

Assignment initialization. The scheduler starts from the fixed primaries and the redundant replicas recorded in the per-layer projected slot table C_t . This table includes bindings established by earlier plans, even if their

copies are still in flight.

To initialize the work assignment, the scheduler divides expert e 's demand into $k_t^e = \lceil d_t^e / M_{\text{tile}} \rceil$ tiles. The tile assignment $N_t = (n_t^{e,w})$ records how many tiles of e run on worker w , whose total load is $T_w = \sum_e n_t^{e,w}$. The scheduler initializes N_t from the preceding assignment A_{t-1} , adapting its per-worker proportions to k_t^e integer tiles while preserving the total. Without a preceding assignment, all tiles start on the expert's primary worker. This step adapts to demand changes while retaining useful assignments from nearby microbatches.

Moving only a few tiles provides limited load relief but can still require additional routing state at the receiver. The scheduler therefore uses a nonnegative integer compactness threshold θ , requiring each rebalancing move to transfer more than θ tiles. Since experts with $0 < k_t^e \leq \theta$ cannot meet this requirement, initialization assigns all their tiles to the primary. Existing replica bindings remain recorded for later reuse.

Rebalancing within the placement. This phase greedily redistributes tiles among replicas already recorded in P . The scheduler scans workers h in decreasing load order and receivers c in increasing load order. For each pair (h, c) , it considers experts with $n_t^{e,h} > 0$ and $c \in \mathcal{R}_e(P)$. If $T_h - T_c > 1$, the scheduler can move

$$q_e(h, c) = \min \left(n_t^{e,h}, \left\lfloor \frac{T_h - T_c}{2} \right\rfloor \right) \quad (10)$$

tiles of expert e from h to c . The two terms ensure that the move has enough tiles and does not reverse the pair's load order. Similarly, only moves with $q_e(h, c) > \theta$ are eligible. For the first pair admitting such a move, the scheduler chooses the expert maximizing $q_e(h, c)$, breaking ties by expert index.

The scheduler applies the move, updates worker loads, and repeats the search until no eligible move remains. Placement P stays fixed, and no weight copy is added to Δ_t . Each move strictly decreases the load vector, sorted from largest to smallest, in lexicographic order. This guarantees termination even when several workers share the maximum load.

Rebalancing beyond the placement. This phase explores whether new replicas can further reduce the FFN bottleneck. It also considers only moves with $q_e(h, c) > \theta$. The search FindBeyondMove follows the preceding phase's greedy rule, but requires $c \notin \mathcal{R}_e(P)$ and a safe redundant slot r on c . Each move updates the current candidate plan by binding slot r to expert e and assigning $q_e(h, c)$ tiles to the replica. This updates C_t and P , replaces any previous binding in r , and adds (e, c, r) to Δ_t .

The scheduler evaluates each updated plan using the net gain in Equation 8. The preceding phase produces the baseline assignment A_t^{base} , giving a fixed FFN time $F_0 = F(A_t^{\text{base}})$. After each move, the scheduler subtracts the candidate's FFN time and exposed migration cost from F_0 . This calculation uses the supplied estimates β_{tile} , $M(\cdot)$, and W_t , accounting for the exposed cost of the entire accumulated copy list Δ_t . Throughout the search, it saves the candidate with the highest positive net gain seen so far. If none is found, it retains the preceding phase's plan. Search can continue through candidates with nonpositive gain, since several moves may be needed to reduce equally loaded bottleneck workers.

Each candidate must satisfy the slot and copy constraints. A global copy budget B limits the total number of new replicas for each target microbatch. The counter b increases by one for each planned copy to a destination worker. For each receiver, the scheduler first chooses an empty redundant slot; otherwise, it chooses the least recently used slot whose replica has no work in N_t . Slots used by the preceding microbatch (U_t) or reserved by an earlier move in this phase cannot be rebound. If no safe slot exists, the receiver is skipped.

The search ends when $b = B$ or no feasible move exceeds the tile threshold. The scheduler then restores the saved placement, assignment, slot bindings, and copy list. All changes during search are tentative: only the selected candidate is recorded and passed to execution.

The scheduler fills the selected tile assignment with tokens, leaving at most one partially filled tile per expert, and returns the resulting plan. It retains the selected assignment and slot bindings for the next microbatch.

6 AFORE Design and Implementation

Section 5 constructs a microbatch plan Π_t for each target microbatch. This section addresses two questions left by that algorithm: how can the target’s routing demand reach the scheduler and the resulting plan reach FFN workers in time, and how can those workers prepare and apply the plan while earlier microbatches are still executing? As shown in Figure 5, AFORE prefetches demand from the attention side, publishes each plan through a shared-memory queue, and overlaps replica migration with preceding FFN computation. We describe these mechanisms in turn.

6.1 Demand Prefetching and Plan Delivery

In a conventional expert-parallel MoE runtime, routing information becomes available only when the FFN workers are about to execute the experts, leaving little time for reconfiguration. AFD provides an earlier opportunity: the attention side produces the hidden states before the corresponding FFN execution. AFORE uses this interval to obtain the target microbatch’s routing demand and initiate placement scheduling, as illustrated in Figure 6.

Extracting routing demand. After producing the hidden states for a MoE layer, each attention worker runs the model’s gate and top- k to obtain the selected expert IDs and routing weights. It counts the IDs to form $d_{t,a}$, where $d_{t,a}^e$ is the number of token–expert assignments to expert e in that worker’s portion of the target microbatch. The IDs and weights are also reused for FFN execution, avoiding a second gate and top- k computation.

A separate path for demand. To deliver the newly formed $d_{t,a}$, StepMesh [25] writes each compact vector directly into a registered host-memory mailbox on the FFN node. The hidden states, expert IDs, and routing weights travel along another path to the FFN workers for execution. This separation makes demand available to the CPU scheduler before the full execution input is assembled, leaving more of the AFD lookahead window for scheduling and subsequent expert migration. Each vector is associated with its serving step, layer, microbatch, and source attention worker, allowing the scheduler to collect demand in target order.

Shared scheduling and plan delivery. Once vectors from all attention workers have arrived for a target microbatch, the shared CPU scheduler forms $d_t = \sum_a d_{t,a}$ and generates Π_t using the algorithm in Section 5.2. It publishes each target-tagged plan once to a shared-memory ring. Because all FFN workers map this ring, one publication makes the same plan available to every worker without separate plan messages. To remain bounded while protecting slow readers, the queue reuses a slot only after all FFN workers have copied its plan. Plan consumption and execution are described in Section 6.2.

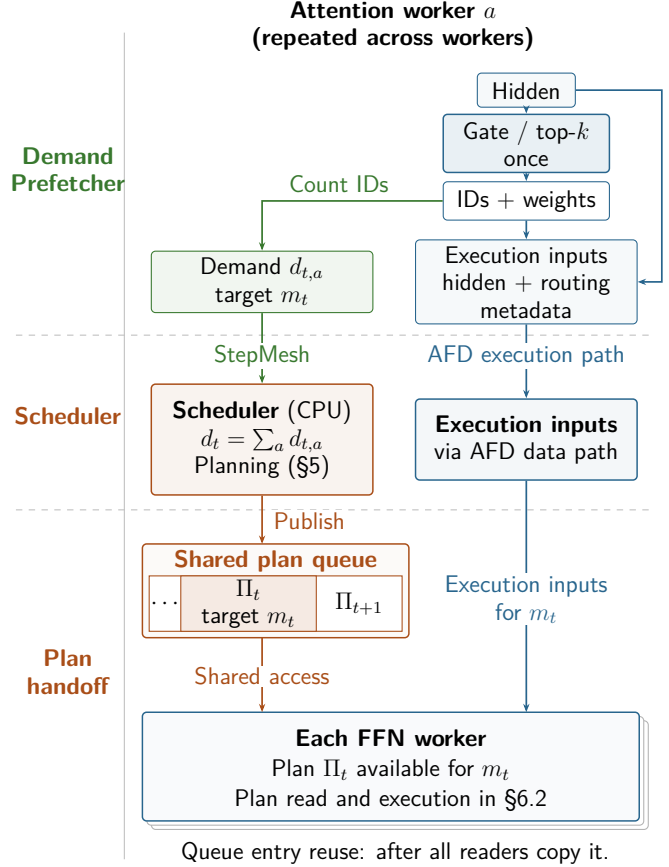


Figure 6 Demand prefetching and plan delivery for m_t , expanding Steps 1–2 in Figure 5. Attention forms routing demand and execution inputs; the CPU scheduler aggregates demand and prepares the plan while inputs follow the AFD path. The shared-memory queue makes the published plan available to all FFN workers. Queue entries are reused after all workers copy them.

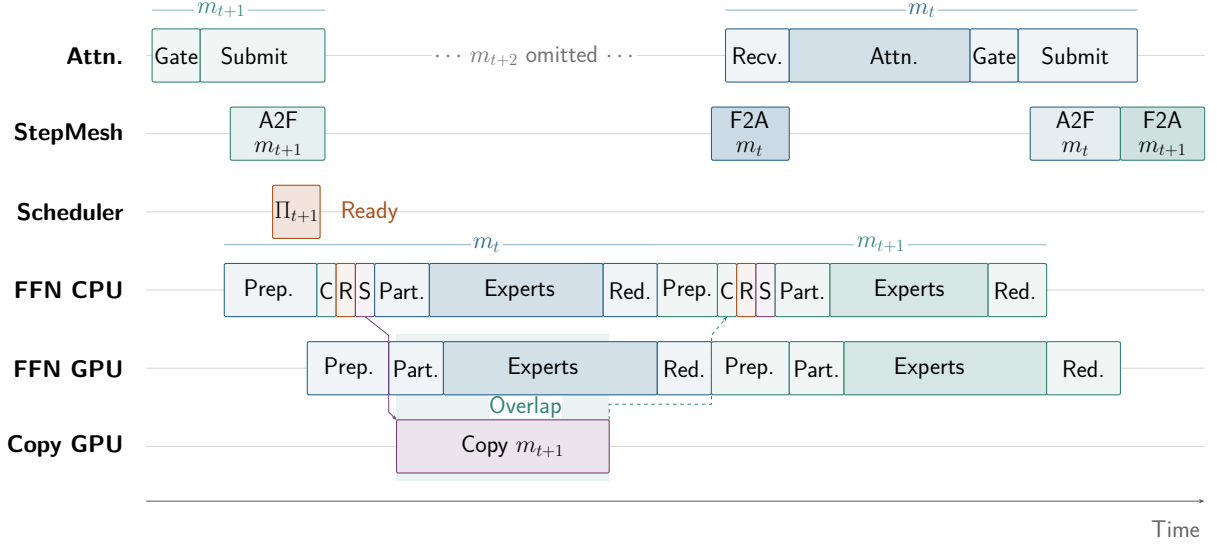


Figure 7 Overlapped migration and plan execution for FFN microbatches m_t and m_{t+1} , expanding Steps 3–4 in Figure 5. Shading covers the copy for m_{t+1} during m_t ’s FFN execution. The first C checks m_t ’s replicas, R reads Π_{t+1} , and S submits its copies. The solid arrow links S to GPU copy execution; the dashed arrow links copy completion to m_{t+1} ’s C, which enqueues a model-stream wait if the copy remains unfinished. The later R/S begins preparation for the following microbatch. A2F/F2A denote StepMesh transfers.

6.2 Placement Migrator and Plan Execution

After the scheduler publishes Π_{t+1} for m_{t+1} to the shared ring, FFN workers must prepare its required replicas before the target can use the plan. This work takes place while m_t is still in flight: weights it uses must remain valid, and new replicas cannot be consumed until their copies complete. Figure 7 shows the handoff across the FFN model thread, model stream, and migration stream.

Plan read and copy submission. Before partitioning m_t ’s token–expert pairs, each FFN worker reads Π_{t+1} from the shared ring (R in Figure 7) and validates its target and sequence to reject a stale or out-of-order plan. It immediately submits the missing copies in Δ_{t+1} (S) to overlap migration with the remainder of m_t ’s execution, while keeping Π_{t+1} pending until m_{t+1} arrives. Replicas already resident in their slots need no further copy.

Overlapped migration. To run the copies alongside m_t ’s FFN computation, the migrator batches these GPU-to-GPU P2P transfers from fixed primary owners to reserved slots on a dedicated migration stream. The source primary may differ from the worker that relinquishes the tiles. Meanwhile, the model thread can continue m_t ’s partition, expert, and reduce/reply stages and prepare later inputs, with their GPU work running on the model stream. To avoid overwriting active replicas, the scheduler excludes slots still used by m_t from the copy destinations. A completion event marks when the copy batch finishes, allowing the target to check replica readiness at C.

Readiness and exact partitioning. When m_{t+1} reaches the FFN stage, workers first all-gather its hidden states and routing metadata received through StepMesh. This gives every worker the same input for exact partitioning. Each worker then retrieves the pending Π_{t+1} and checks the required replicas’ copy-completion events (C in Figure 7). Completed copies require no wait; for any still in flight, the worker enqueues a model-stream wait before using the new replica. With the weights ready, workers partition the target’s token–expert pairs according to A_{t+1} and σ_{t+1} . The scheduler’s integer count $a_{t+1}^{e,w}$ specifies how many of expert e ’s pairs execute on worker w . Because all FFN workers share the gathered routing result, they can number each occurrence of e in the same order and select disjoint intervals without another token transfer.

For selected redundant workers $w_1, \dots, w_{k_e}$ in rank order, the cumulative boundaries are

$$o_0 = 0, \quad o_j = o_{j-1} + a_{t+1}^{e, w_j}. \quad (11)$$

Worker w_j executes ordinals in $[o_{j-1}, o_j)$, and the fixed primary owner executes the remainder $[o_{k_e}, d_{t+1}^e)$. These disjoint intervals realize A_{t+1} exactly. Each worker masks pairs outside its interval and maps retained logical expert IDs to fixed primary slots or redundant slots specified by σ_{t+1} , preserving the original routing weights. The model all-reduce combines partial outputs before StepMesh returns the corresponding slices to attention.

7 System Evaluation

We evaluate the design and implementation of AFORE by answering the following research questions:

- **(RQ1)** How does AFORE compare against state-of-the-art baselines in output throughput and P95 inter-token latency?
- **(RQ2)** How efficient and effective is AFORE’s scheduling algorithm in practice?
- **(RQ3)** How much does each system design contribute to AFORE’s performance gains?
- **(RQ4)** How effectively does AFD pipeline overlap hide expert-migration overhead?

7.1 Experimental Setup

Hardware environment. We conduct all experiments on a cluster of two machines, each equipped with eight NVIDIA A100 GPUs. Within each node, the eight GPUs are interconnected through NVLink and NVSwitch, providing a peak unidirectional bandwidth of 300 GB/s per GPU. For inter-node communication, each node provides eight RDMA-enabled bonded network interfaces with 50 GB/s of unidirectional bandwidth per bond.

Models and workloads. We evaluate AFORE on a 110B-parameter MoE model (i.e., GLM-4.5-Air), served under a PD+AFD deployment. We construct four representative serving workloads from ShareGPT [31], FineWeb [19], CodeForces [20], and GSM8K [3]. These workloads are diverse, covering conversational requests, document-style inputs, code-generation tasks, and mathematical-reasoning queries, respectively. We replay the traces to generate dynamic serving workloads whose expert-token distributions shift over time.

Evaluation metrics. We report standard LLM serving metrics. We measure end-to-end output throughput in tokens per second and P95 inter-token latency (ITL), which captures the tail latency of decode-token generation. For migration-specific analysis, we report raw migration cost and exposed migration cost on the target microbatch’s critical path.

Baselines. We compare AFORE against representative expert-placement and reconfiguration policies. For a fair comparison, we implement all baselines inside the same AFD serving runtime and run them with the same model, hardware, GPU allocation, batching policy, and workloads:

- **Static**, a fixed-placement baseline that evenly assigns experts to FFN workers at initialization and never changes the placement during serving.
- **EPLB** [6], a statistics-based reconfiguration policy that periodically rebalances expert placement using load statistics collected over a past profiling window.
- **HarMoEny** [7], a reactive load-balancing policy that redistributes tokens and creates redundant expert replicas based on the routing demand observed at runtime.
- **Lina** [16], a prediction-based reconfiguration policy that predicts upcoming hot experts from recent expert selection patterns and replicates them in advance.

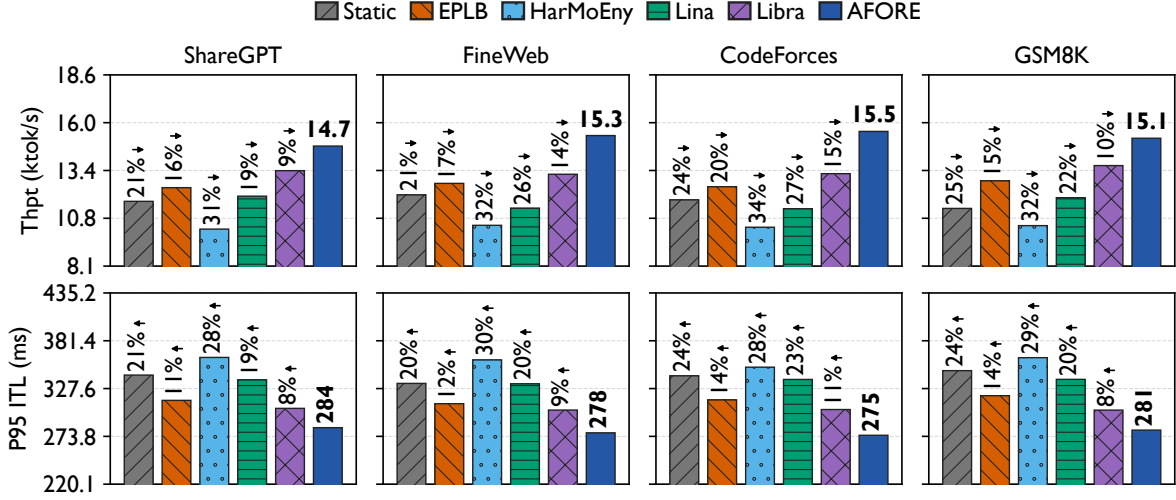


Figure 8 Upper: Output throughput across four workloads. Lower: P95 inter-token latency under the same workloads.

- **Libra** [30], a prediction-based reconfiguration policy that prefetches expert replicas based on predicted next-layer top- k activations and redistributes tokens according to the actual routing demand.

We further evaluate ablated versions of AFORE in §7.4 to isolate the contributions of target-demand visibility and pipeline-overlapped migration.

7.2 End-to-end Performance (RQ1)

Figure 8 compares AFORE with the baselines across the four workloads. AFORE achieves the highest output throughput and the lowest P95 ITL on all workloads. Compared with the strongest competing baseline for each workload, AFORE improves throughput by 10.1–17.6% and reduces P95 ITL by 7.1–9.5%.

The improvement comes from two effects. First, temporal reconfiguration reduces FFN-side stragglers by adapting expert placement to the routing demand that is about to enter the FFN stage. Second, AFORE overlaps expert migration with the AFD pipeline, avoiding the critical-path migration cost that limits more reactive policies. As a result, AFORE improves both throughput and tail token latency.

The gains persist across workloads, i.e., conversational (ShareGPT), document-style (FineWeb), coding (CodeForces), and mathematical-reasoning (GSM8K) workloads. Despite their different request characteristics and routing patterns, AFORE consistently improves the MoE serving efficiency, showing that temporal expert reconfiguration is robust across diverse workload types.

7.3 Scheduler Overhead and Solution Quality (RQ2)

Evaluation configuration. We evaluate whether AFORE’s scheduler is efficient enough to run at microbatch granularity while producing high-quality plans. We replay routing traces from the four workloads under EP degrees 2, 4, 8, 16, 32, 64, and 128, sampling the same 200 steady-state targets (50 per workload) at each EP degree. For scheduling overhead, we report the median scheduler core latency across the sampled targets. For solution quality, we compare AFORE’s maximum worker tile load with the optimum under the same placement and copy constraints. Optimality is certified by matching the average tile-load lower bound or by exact optimization under the same constraints. We report the relative gap between AFORE’s maximum worker tile load and the optimum.

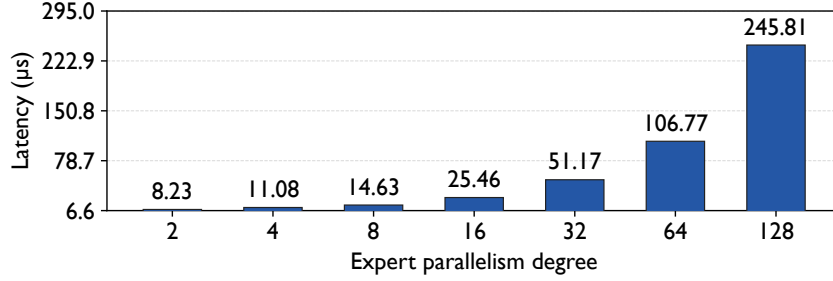


Figure 9 Median scheduler core latency of AFORE across EP degrees.

Table 2 Scheduler solution quality. Gaps are relative to the optimum.

EP	2	4	8	16	32	64	128
Optimal / 200	196	197	200	200	200	200	200
Mean gap (%)	0.04	0.03	0	0	0	0	0
Max gap (%)	3.60	1.89	0	0	0	0	0

Scheduling overhead. As shown in Figure 9, AFORE’s scheduler adds only sub-millisecond planning overhead across all tested EP degrees. Its median latency is 8.23μ s at EP=2 and remains 245.81μ s even at EP=128. This low planning cost makes the scheduler suitable for microbatch-level reconfiguration as the number of workers increases.

Solution quality. Table 2 compares AFORE’s greedy scheduler with the certified optimum in maximum worker tile load. At EP=2 and EP=4, the mean optimality gap is at most 0.04%, with a worst-case gap of 3.60% across the two settings. At every tested EP degree from 8 to 128, AFORE achieves the optimum on all 200 targets. Overall, it achieves the optimal maximum worker tile load on 99.5% of the 1,400 targets.

These results show that AFORE’s scheduler provides an effective trade-off for online serving: it combines the low latency needed for frequent reconfiguration with near-optimal load balance, even at high EP degrees.

7.4 Component-wise Ablation (RQ3)

Evaluation configuration. We isolate the effects of demand prefetching and migration overlap. *Full AFORE* enables both mechanisms. *w/o Prefetch* replaces the target microbatch’s demand with the previous step’s demand at the same layer and microbatch position, while retaining the same scheduler and migration mechanism. *w/o Overlap* retains demand prefetching and the same scheduler, but places expert migration on the critical path. *Static* disables runtime reconfiguration and serves as a fixed-placement reference. All configurations use the same model, workload, resource allocation, and batching policy.

End-to-end contribution. Figure 10 reports output throughput and P95 inter-token latency across the four workloads. Compared with Static, full AFORE improves throughput by 26.0–34.1% (29.8% on average) and reduces P95 ITL by 16.7–19.6% (18.2% on average). The combined design consistently improves serving performance over fixed expert placement.

Demand prefetching. Disabling demand prefetching reduces throughput by 12.4–17.9% and increases P95 ITL by 6.3–10.5% relative to full AFORE. Without the target microbatch’s routing demand, the scheduler acts on historical observations and may optimize for a load distribution that has already changed. The degradation across workloads illustrates the value of timely routing information for expert reconfiguration.

Table 3 Ablation study configurations.

Configuration	Reconfig.	Prefetch	Overlap
Static	–	–	–
w/o Prefetch	✓	–	✓
w/o Overlap	✓	✓	–
AFORE	✓	✓	✓

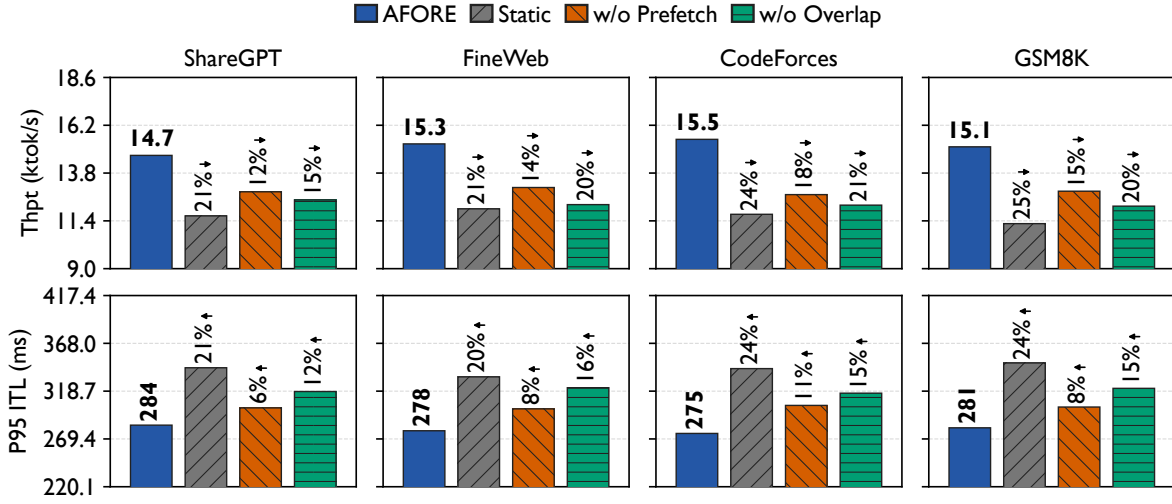


Figure 10 Upper: Output throughput for the component-wise ablation across four workloads. Lower: P95 inter-token latency under the same configurations.

Migration overlap. Without pipeline overlap, throughput drops by 15.2–21.3%, while P95 ITL rises by 12.2–16.0%. Delaying expert copies exposes their latency on the target microbatch’s critical path, even though the scheduler still uses its actual routing demand.

Therefore, demand prefetching and migration overlap play complementary roles: prefetching improves *which* placement is selected, while overlap reduces the cost of *installing* that placement.

7.5 Migration Hiding Effectiveness (RQ4)

Evaluation configuration. We measure the raw migration cost from copy submission to completion and the exposed migration cost as the remaining wait for copies when the target microbatch needs them. Their difference is the migration time hidden by pipeline overlap. We also measure the lookahead window from copy submission to the target microbatch’s first use of the replicas.

Migration cost. The left panel of Figure 11 compares raw migration cost with the lookahead window across the four workloads. Mean raw migration cost ranges from 0.453–0.495 ms, compared with a mean lookahead window of 2.008–2.119 ms. All 324 sampled targets complete their required copies before use, yielding zero exposed migration cost. Thus, pipeline overlap fully hides the migration time in these samples and keeps it off the target microbatch’s critical path.

Time remaining before use. The right panel of Figure 11 reports the time between copy completion and first use, taking the minimum across participating ranks for each target. The median remaining time ranges from 1.459–1.668 ms across workloads, with a minimum of 0.447 ms. This remaining window allows additional migration delay to be absorbed before it becomes exposed on the target microbatch’s critical path.

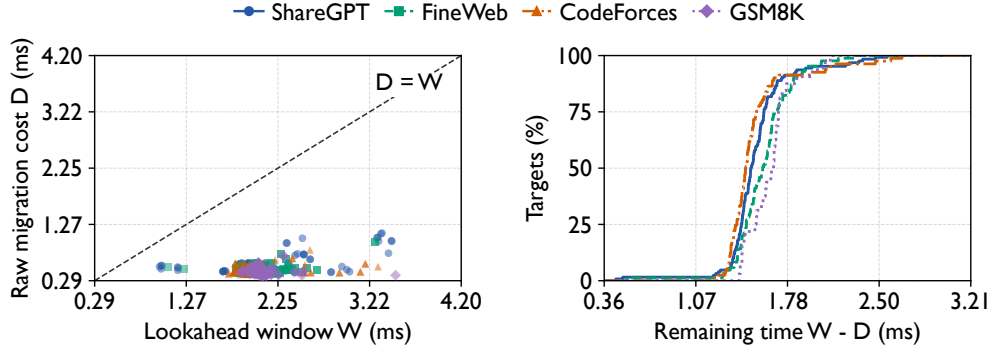


Figure 11 Left: Raw migration cost D versus lookahead window W . Right: CDF of time remaining before use (minimum across ranks per target).

8 Conclusion

In this paper, we presented AFORE, a timely expert reconfiguration system for attention-FFN disaggregated MoE serving. AFORE addresses the FFN-side load imbalance amplified by AFD by exploiting two properties of the disaggregated pipeline: upcoming microbatch demand is visible before FFN execution, and expert migration can make progress while preceding microbatches are still executing. Based on these key observations, AFORE formulates expert reconfiguration as a microbatch-aware scheduling problem that jointly considers near-future expert demand, FFN-stage bottlenecks, and exposed migration overhead. AFORE’s scheduler uses a primary-plus-redundant expert layout, a migration-aware cost model, and greedy load redistribution to decide when to install redundant expert replicas. At runtime, attention workers send compact vectors derived from the target routing directly to the scheduler. AFORE then installs missing replicas with GPU-to-GPU transfer and applies routing updates at microbatch boundaries without disrupting in-flight execution. Our evaluation shows that timely expert reconfiguration improves end-to-end serving efficiency across diverse workloads. Compared with the strongest baseline, AFORE improves output throughput by 10.1–17.6% and reduces P95 inter-token latency by 7.1–9.5%. The scheduler remains lightweight enough for microbatch-level operation, with sub-millisecond latency even at high expert-parallelism degrees. Component-wise ablation isolates the benefits of true target-microbatch demand and early copy submission. Migration profiling then separates raw migration cost from exposed migration cost and quantifies how often the AFD lookahead window hides replica installation. These results demonstrate that AFD exposes not only a new bottleneck for MoE serving, but also the temporal structure needed to mitigate it efficiently.

References

- [1] Oana Balmau, Anne-Marie Kermarrec, Rafael Pires, André Loureiro Espírito Santo, Martijn de Vos, and Milos Vujanovic. Accelerating moe model inference with expert sharding. In *Proceedings of the 5th Workshop on Machine Learning and Systems*, pages 192–199, 2025.
- [2] Shaoyuan Chen, Wencong Xiao, Yutong Lin, Mingxing Zhang, Yingdi Shan, Jinlei Jiang, Kang Chen, and Yongwei Wu. Efficient heterogeneous large language model decoding with model-attention disaggregation. *arXiv preprint arXiv:2405.01814*, 2024.
- [3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [4] Peizhuang Cong, Aomufei Yuan, Shimao Chen, Yuxuan Tian, Bowen Ye, and Tong Yang. Prediction is all moe needs: Expert load distribution goes from fluctuating to stabilizing. *arXiv preprint arXiv:2404.16914*, 2024.
- [5] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. In

- Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1280–1297, 2024.
- [6] DeepSeek AI. Expert parallelism load balancer (eplb), 2025.
 - [7] Zachary Doucet, Rishi Sharma, Martijn de Vos, Rafael Pires, Anne-Marie Kermarrec, and Oana Balmau. Harmoeny: Efficient multi-gpu inference of moe models. *arXiv preprint arXiv:2506.12417*, 2025.
 - [8] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. Glam: Efficient scaling of language models with mixture-of-experts. In *International conference on machine learning*, pages 5547–5569. PMLR, 2022.
 - [9] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
 - [10] Elias Frantar and Dan Alistarh. Qmoe: Practical sub-1-bit compression of trillion-parameter models. *arXiv preprint arXiv:2310.16795*, 2023.
 - [11] Seokjin Go and Divya Mahajan. Moetuner: Optimized mixture of expert serving with balanced expert placement and token routing. *arXiv preprint arXiv:2502.06643*, 2025.
 - [12] Jiaao He, Jiezhong Qiu, Aohan Zeng, Zhilin Yang, Jidong Zhai, and Jie Tang. Fastmoe: A fast mixture-of-expert training system. *arXiv preprint arXiv:2103.13262*, 2021.
 - [13] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems*, 5:269–287, 2023.
 - [14] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
 - [15] Dmitry Lepikhin, Hyukjoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
 - [16] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed MoE training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 945–959, 2023.
 - [17] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
 - [18] NVIDIA. Scaling large moe models with wide expert parallelism on nvl72 rack-scale systems. <https://developer.nvidia.com/blog/scaling-large-moe-models-with-wide-expert-parallelism-on-nvl72-rack-scale-systems/>, 2025.
 - [19] Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=n6SCkn2QaG>.
 - [20] Guilherme Penedo, Anton Lozhkov, Hynek Kydlicek, Loubna Ben Allal, Edward Beeching, Agustin Piqueres Lajarin, Quentin Gallouedec, Nathan Habib, Lewis Tunstall, and Leandro von Werra. Codeforces. <https://huggingface.co/datasets/open-r1/codeforces>, 2025.
 - [21] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*, pages 18332–18346. PMLR, 2022.
 - [22] Gursimran Singh, Timothy Yu, Haley Li, Cheng Chen, Hanieh Sadri, Qintao Zhang, Yu Zhang, Ying Xiong, Yong Zhang, and Zhenan Fan. Elasticmoe: An efficient auto scaling method for mixture-of-experts models. *arXiv preprint arXiv:2510.02613*, 2025.
 - [23] Chendong Song, Meixuan Wang, Hang Zhou, Hong Liang, Yuan Lyu, Zixi Chen, Yuwei Fan, and Zijie Zhou. Theoretically optimal attention/ffn ratios in disaggregated llm serving. *arXiv preprint arXiv:2601.21351*, 2026.

- [24] vLLM Ascend Team. Expert load balance (eplb): Swiftbalancer. https://docs.vllm.ai/projects/ascend/en/main/user_guide/feature_guide/eplb_swift_balancer.html, 2025.
- [25] Bin Wang, Bojun Wang, Changyi Wan, Guanzhe Huang, Hanpeng Hu, Haonan Jia, Hao Nie, Mingliang Li, Nuo Chen, Siyu Chen, et al. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding. *arXiv preprint arXiv:2507.19427*, 2025.
- [26] Shaoyu Wang, Guangrong He, Geon-Woo Kim, Yanqi Zhou, and Seo Jin Park. Toward cost-efficient serving of mixture-of-experts with asynchrony. *arXiv preprint arXiv:2505.08944*, 2025.
- [27] Tian Wu, Liming Wang, Zijian Wen, Xiaoxi Zhang, Jingpu Duan, Xianwei Zhang, and Jinhang Zuo. Accelerating edge inference for distributed moe models with latency-optimized expert placement. *arXiv preprint arXiv:2508.12851*, 2025.
- [28] Yize Wu, Ke Gao, Ling Li, and Yanjun Wu. Easybalance: Cross-layer load balancing in distributed MoE inference. *arXiv preprint arXiv:2608.07964*, 2026.
- [29] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [30] Jaehoon Yang, Yushin Kim, Seokwon Moon, Yeonhong Park, and Jae W. Lee. Libra: Effective yet efficient load balancing for large-scale MoE inference. In *International Conference on Learning Representations*, 2026.
- [31] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.
- [32] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, et al. Megascale-infer: Serving mixture-of-experts at scale with disaggregated expert parallelism. *arXiv preprint arXiv:2504.02263*, 2025.